Rule Engine Formalization

Structural Inspection Specification for ID_3 Freeze Discipline

APOD-ID3-RULE-001

Document Control Information

Document Identifier: APOD-ID3-RULE-001
Title: Access-Mode ID_3 — Rule Engine Formalization
Edition: Access-Mode ID_3 — Operator / Inspection & Freeze Discipline
Date of Issue: 28 February 2026
Status: Informative Companion — Non-Normative
Author: Anthony Siljeg
Affiliation: Alnotrea Labs / Policy of Developments — Access PoD *Initiative*

Abstract

The Access-Mode ID_3 Rule Engine formalizes deterministic structural inspection enforcement for artifacts prepared under ID_3 freeze discipline.

This specification defines:

- Structural constraint categories
- Validator taxonomy
- Enforcement severity classification
- Non-fabrication invariants
- Deterministic correction logic
- Canonical promotion and freeze eligibility conditions
- Deterministic integrity anchoring requirements
- Rule versioning and extension governance

The Rule Engine operates exclusively within ID_3 inspection posture. It enforces structural correctness prior to canonical promotion and freeze.

It does not evaluate governance adequacy, compliance status, policy sufficiency, or decision validity.

This document defines the formal structural enforcement model underlying the machine readable rule specification in: `\rules\session_completeness.rule.json`

Authority remains external to the Rule Engine.

Specification Positioning Note (Informative)

This release is separate from the ID_3 Operator release.

The **Access-Mode ID_3 Operator** release contains:

- Practical execution workflow (Inspect → Freeze → Hash)
- Inspection scripts and toolkit examples
- Canonical promotion and integrity anchoring mechanics

This document — **Rule Engine Formalization (APOD-ID3-RULE-001)** — defines the structural inspection specification underlying those mechanics.

It serves as:

- The formal constraint model for `session_completeness.rule.json`
- A deterministic validation and enforcement specification
- A versioned structural governance artifact
- A versioned structural enforcement specification that may expand independently within the Access PoD ID_3 architecture.

Future revisions of this specification may:

- Introduce additional validator types
- Refine constraint categories
- Expand module coverage
- Accept contributor input under version-controlled change governance

This document defines structural enforcement semantics.
Operational execution guidance remains in the ID_3 Operator release:

https://github.com/Access-PoD/access-pod-artifacts/releases/tag/ID_3-operator

Contents

Abstract 1

Specification Positioning Note (Informative) 2

1. Executive Overview 6

2. Architectural Positioning 6

3. Rule Classification Model 7

3.1 Required Structural Constraints 7

3.2 Optional Structural Signals 8

3.3 Non-Fabrication Constraints 8

4. Pseudo-Schema Model 9

5. Enforcement Flow Model 10

6. Diagnostic Scoring Model 10

7. Validator Taxonomy 10

7.1 Validator Classes 10

A. Presence Validators 11

B. Type Validators 11

C. Structural Shape Validators 11

D. Semantic Prohibition Validators 12

E. Placeholder Detection Validators 12

8. Enforcement Severity Model 13

8.1 Severity Levels 13

8.2 Enforcement Logic 13

9. Non-Fabrication Boundary Model 13

9.1 Principle 13

9.2 Formal Non-Fabrication Constraints 14

9.3 Correction Eligibility Matrix 14

9.4 Escalation Trigger 14

10. Deterministic Correction Logic Model 15

10.1 Correction Phases 15
10.2 Deterministic Normalization Rules 15
10.3 Canonical Determinism Invariant 15
11. Enforcement State Machine 16
12. Deterministic Hash Algorithm Declaration 16
12.1 Algorithm Standard 16
12.2 Canonical Hash Invariant 17
13. Conformance Language Model 17
14. Rule Execution Contract 18
14.1 Input Contract 18
14.2 Execution Order 18
15. Correction Safety Model 18
15.1 Allowed Mutations 18
15.2 Disallowed Mutations 18
16. Error Classification Model 19
17. Rule Versioning and Extension Governance 19
17.1 Version Identifier Discipline 19
17.2 Extension Categories 20
17.3 Backward Compatibility Model 20
18. Structural Non-Authority Statement 21
Annex A — Terminology Alignment (Informative) 21
A.1 Purpose 21
A.2 Core Terms 21
Rule Engine 21
Required Structural Constraint 22
Optional Structural Signal 22
Non-Fabrication Constraint 22
Canonical Artifact 22
Freeze 22

Integrity Anchor 22
Completeness Score 23
A.3 Alignment with ISO/IEC Governance Language 23
Annex B — Validator Definitions (Informative) 23
B.1 Validator Type Registry 23
presence 23
type 24
shape 24
placeholder_check 24
prohibition 24
B.2 datetime_utc Validator Specification 25
B.3 Deterministic Ordering Rule 25
Annex C — Algorithm References (Informative) 25
C.1 Hashing Algorithm 25
C.2 Canonicalization Requirements 26
Annex D — Worked Example (Baseline vs Variant Enforcement Trace) 26
D.1 Scenario Overview 26
D.2 Initial Verification 26
Baseline 26
Variant 27
D.3 Correction Phase 27
D.4 Canonical Promotion 28
D.5 Freeze and Hash 28
D.6 Observations 28

1. Executive Overview

The ID_3 Rule Engine formalizes structural inspection discipline for session artifacts prepared under Access-Mode ID_3.

It governs:

- Structural completeness validation
- Metadata normalization boundaries
- Canonical promotion eligibility
- Freeze eligibility conditions
- Integrity Anchor generation

The Rule Engine does not evaluate governance adequacy, compliance status, policy sufficiency, or decision validity.

It enforces structural correctness only.

The specification operates as:

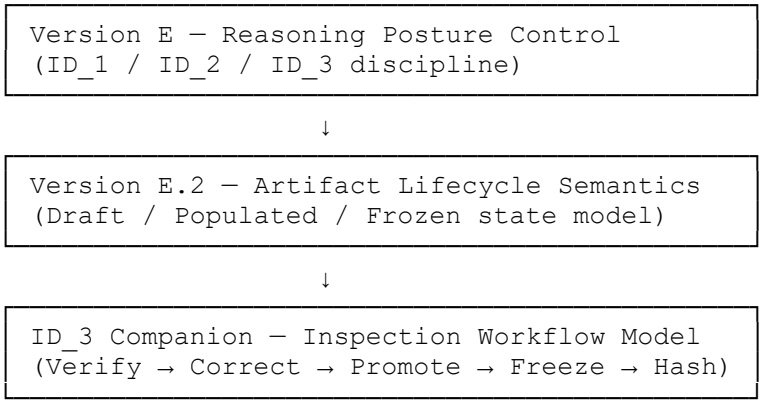
- A machine-readable JSON rule definition
- A deterministic validation model
- A posture-bound enforcement layer
- A diagnostic scoring mechanism

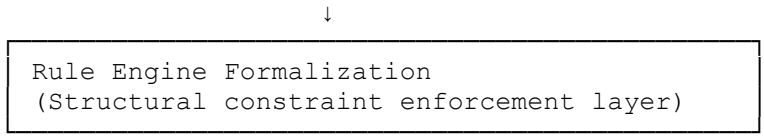
This document defines the formal model underlying that specification.

---

2. Architectural Positioning

The Rule Engine exists within the Access-Mode layered architecture:





The Rule Engine:

- Operates only under ID_3 posture
- Enforces structural alignment prior to freeze
- Prevents silent schema drift
- Preserves non-fabrication discipline

It is subordinate to reasoning posture discipline and artifact lifecycle semantics.

---

3. Rule Classification Model

The Rule Engine organizes constraints into three formal categories:

1. **Required Structural Constraints**
2. **Optional Structural Signals**
3. **Non-Fabrication Constraints**

Each category has different enforcement semantics.

3.1 Required Structural Constraints

Required constraints:

- Must validate successfully before canonical promotion
- Block freeze if violated
- Enforce structural integrity only

Examples:

- `schema_version` presence
- Valid `session.session_id`
- ISO 8601 UTC timestamps
- Required nested object structure
- Mandatory top-level keys

Required constraints are binary: PASS enables progression; FAIL blocks canonical promotion.

They do not measure quality.

3.2 Optional Structural Signals

Optional constraints:

- Contribute to diagnostic completeness scoring
- Do not block freeze
- May highlight traceability gaps

Examples:

- `facilitator.name` present
- `participants.role` populated
- `run_plan.notes` populated
- `signoff.signed_by` populated

Optional signals are advisory.

They support transparency, not authority.

Completeness scoring must never:

- Drive content invention
- Trigger synthetic population
- Override non-fabrication boundaries

3.3 Non-Fabrication Constraints

The Rule Engine enforces a strict non-fabrication boundary.

Structural correction is permitted.
Substantive creation is prohibited.

A formal invariant model defining non-fabrication constraints is provided in Section 9.

---

4. Pseudo-Schema Model

The Rule Engine specification is expressed as structured JSON.

Below is the conceptual schema model.

```
{
  "rule_id": "CSC_SESSION_COMPLETENESS_V1_1",
  "schema_version": "rule_v1_1",
  "intended_use": "Diagnostic completeness scoring only. Must not drive
content edits beyond minimal metadata correction prior to hash/freeze.",
  "categories": {
    "required": {
      "session.schema_version": { "type": "string", "required": true },
      "session.session_id": { "type": "string", "required": true },
      "session.created_utc": { "type": "datetime_utc", "required": true }
    },
    "optional": {
      "session.facilitator.name": { "type": "string", "weight": 2 },
      "session.participants": { "type": "array", "weight": 3 }
    },
    "non_fabrication": {
      "no_inferred_fields": true,
      "no_filesystem_timestamp_derivation": true,
      "no_synthetic_content_generation": true
    }
  },
  "scoring": {
    "max_points": 47,
    "diagnostic_only": true
  }
}
```

This pseudo-schema illustrates:

- Required vs optional separation
- Explicit non-fabrication rules
- Diagnostic scoring semantics
- Explicit intended use boundary

The rule file governs enforcement behavior.

This document governs its conceptual interpretation.

---

5. Enforcement Flow Model

The Rule Engine operates within a deterministic enforcement sequence:

```
Intake Surface (10_working)
        ↓
Initial Verification
        ↓
Structural Correction (20_corrected)
        ↓
Re-Verification
        ↓
Canonical Promotion Eligibility Check
        ↓
Freeze Eligibility (prior to Freeze Declaration)
```

---

6. Diagnostic Scoring Model

Completeness scoring:

- Aggregates optional field presence
- Assigns weighted values
- Produces percentage completeness

Scoring is explicitly:

- Diagnostic only
- Non-evaluative
- Non-authoritative
- Non-maturity signaling

Completeness scoring is diagnostic and does not determine freeze eligibility.

Scoring supports inspection transparency only.

---

7. Validator Taxonomy

7.1 Validator Classes

The Rule Engine employs structured validators grouped by functional class.

A. Presence Validators

Confirm required keys exist.

```
{
  "validator_type": "presence",
  "field": "session.schema_version",
  "required": true
}
```

Failure outcome: `ERROR`

B. Type Validators

Confirm JSON value type.

Supported types:

- `string`
- `number`
- `boolean`
- `array`
- `object`
- `datetime_utc`
- `enum`
- `regex_match`

Example:

```
{
  "validator_type": "type",
  "field": "session.created_utc",
  "expected_type": "datetime_utc"
}
```

C. Structural Shape Validators

Ensure array/object structure conforms.

Example:

```
{
  "validator_type": "shape",
  "field": "session.participants",
  "must_be_array": true
}
```

If scalar detected → coercion permitted (metadata-bound).

D. Semantic Prohibition Validators

Detect disallowed patterns.

Example:

```
{
  "validator_type": "prohibition",
  "field": "signoff.signed_at_utc",
  "condition": "derived_from_filesystem"
}
```

Failure outcome: `ESCALATION_REQUIRED`

E. Placeholder Detection Validators

Detect non-final identity markers.

```
{
  "validator_type": "placeholder_check",
  "field": "session.session_id",
  "pattern": "ID2-YYYYMMDD-HHMM-<shortname>"
}
```

Correction permitted only via deterministic mapping.

---

8. Enforcement Severity Model

Each rule violation is classified under a strict severity taxonomy.

8.1 Severity Levels

Severity	Meaning	Freeze Impact
ERROR	Required structural violation	Freeze blocked
WARNING	Optional structural deficiency	Freeze permitted
INFO	Informational condition	No impact
ESCALATION_REQUIRED	Non-fabrication boundary risk	Revert to ID_2

Severity classification is deterministic and bound to rule definitions. It cannot be overridden during execution.

8.2 Enforcement Logic

```
IF severity = ERROR
    THEN block canonical promotion.

IF severity = ESCALATION_REQUIRED
    THEN halt inspection cycle and revert to ID_2 posture.

IF severity = WARNING
    THEN log diagnostic only.

IF severity = INFO
    THEN record without impact.
```

---

9. Non-Fabrication Boundary Model

9.1 Principle

The Rule Engine operates under a strict non-fabrication constraint.

Structural correction is permitted.
Substantive creation is prohibited.

The boundary prevents inspection posture from becoming reasoning posture.

9.2 Formal Non-Fabrication Constraints

The engine must enforce the following invariant conditions:

```
INVARIANT_1: No new semantic content may be introduced.
INVARIANT_2: No missing required reasoning fields may be inferred.
INVARIANT_3: No timestamps may be derived from filesystem metadata.
INVARIANT_4: No identity values may be constructed without deterministic
source.
INVARIANT_5: Optional emptiness must remain emptiness.
```

Violation of any invariant must:

- Halt correction,
- Prevent canonical promotion,
- Require reversion to ID_2 posture.

9.3 Correction Eligibility Matrix

The following matrix defines what the engine may and may not modify:

Field Type	Structural Edit Allowed	Semantic Edit Allowed
Placeholder ID	Yes (deterministic)	No
Timestamp format	Yes (format only)	No
Array vs scalar shape	Yes	No
Missing reasoning field	No	No
Empty optional field	No	No
Decision record	No	No
Sign-off metadata (missing)	No	No

The Rule Engine is a structural validator, not a governance repair system.

9.4 Escalation Trigger

If correction would require semantic inference:

```
ESCALATION_CONDITION:
    Structural alignment cannot be achieved without altering reasoning
substance.
```

Then:

```
ACTION:
    Revert artifact to ID_2 posture.
    Initiate new operational reasoning session.
    Generate new inspection cycle upon completion.
```

Inspection MUST NOT overwrite reasoning substance.

---

10. Deterministic Correction Logic Model

10.1 Correction Phases

Correction logic operates in bounded phases:

```
Phase 1 — Shape Validation
Phase 2 — Deterministic Normalization
Phase 3 — Structural Re-Validation
```

Each phase must complete successfully before promotion eligibility.

10.2 Deterministic Normalization Rules

Normalization must satisfy:

```
RULE_A: All datetime fields → ISO 8601 UTC.
RULE_B: All scalar-to-array coercions preserve original value.
RULE_C: JSON object key ordering deterministic.
RULE_D: Placeholder identifiers replaced only via declared source.
RULE_E: No field addition beyond schema-defined keys.
```

Determinism requires that:

Given identical corrected input → identical canonical output.

10.3 Canonical Determinism Invariant

Let:

- C = corrected artifact set
- F = canonical artifact set

- H = hash surface

Then:

$$\text{IF } C_1 = C_2$$
$$\text{THEN } F_1 = F_2$$
$$\text{AND } H_1 = H_2$$

Violation of this invariant blocks freeze eligibility.

---

11. Enforcement State Machine

The Rule Engine operates as a finite-state inspection machine.

```
[UNVERIFIED]
      ↓ verify
[STRUCTURAL_VALIDATED]
      ↓ correct
[CORRECTED]
      ↓ re-verify
[ELIGIBLE_FOR_PROMOTION]
      ↓ canonicalize
[CANONICAL_PROMOTED]
      ↓ freeze
[FROZEN_UNDER_ID3]
      ↓ hash
[INTEGRITY_ANCHORED]
```

Transitions are one-directional within an inspection cycle.

Reversal requires:

- New cycle initiation
- Explicit version increment

---

12. Deterministic Hash Algorithm Declaration

12.1 Algorithm Standard

ID_3 freeze integrity anchoring uses:

SHA-256 cryptographic hashing

Requirements:

- Canonical JSON must be fully normalized before hashing.
- Hash order must follow deterministic file ordering.
- No runtime metadata may influence hash content.

12.2 Canonical Hash Invariant

Let:

- F = canonical artifact set
- H = SHA-256(F)

Then:

$$\text{IF } F_1 = F_2 \quad \text{THEN } H_1 = H_2$$

Violation indicates non-deterministic canonical promotion and invalidates freeze eligibility.

---

13. Conformance Language Model

This specification adopts constrained conformance terminology:

Term	Meaning
MUST	Required for freeze eligibility
MUST NOT	Prohibited under all conditions
SHOULD	Recommended structural practice
MAY	Permitted under rule constraints

Examples:

- Required structural fields MUST validate before canonical promotion.
- The Rule Engine MUST NOT generate semantic content.
- Optional completeness scoring SHOULD be logged.
- Deterministic ordering MAY be implemented via stable key sort.

Conformance language applies to structural enforcement only.

---

14. Rule Execution Contract

The Rule Engine operates under a fixed execution contract.

14.1 Input Contract

Input must be:

- JSON artifact set
- Conforming to declared schema version
- Derived from ID_2 posture

If schema_version missing → ERROR.

14.2 Execution Order

```
1. Load rule specification
2. Validate required fields
3. Validate structural shape
4. Detect placeholders
5. Apply deterministic corrections
6. Re-validate required constraints
7. Compute completeness score
8. Return enforcement status
```

Execution order MUST NOT vary.

---

15. Correction Safety Model

Correction operations are strictly bounded.

15.1 Allowed Mutations

- Scalar → array coercion
- Datetime format normalization
- Deterministic ID replacement
- Canonical ordering normalization

15.2 Disallowed Mutations

- Insertion of absent semantic fields
- Removal of recorded gaps
- Reconstruction of decision content
- Timestamp fabrication
- Content rewording

15.3 Correction Invariant

Correction MUST preserve reasoning substance, remain metadata-bound, and be reproducible. Correction MUST NOT introduce semantic content.

---

16. Error Classification Model

Rule violations are recorded in structured form:

```
{
  "violation_id": "VAL_001",
  "severity": "ERROR",
  "field": "session.schema_version",
  "description": "Missing required schema_version",
  "correctable": false
}
```

Correctable flag determines eligibility for automatic correction.

If `correctable = false` and severity ≥ ERROR → freeze blocked.

---

17. Rule Versioning and Extension Governance

17.1 Version Identifier Discipline

Each rule specification MUST include:

```
{
  "rule_id": "CSC_SESSION_COMPLETENESS_V1_1",
  "schema_version": "rule_v1_1",
  "intended_use": "Diagnostic completeness scoring only."
}
```

Rule modification requires:

- rule_id increment
- schema_version increment
- change log documentation
- backward compatibility review
- non-fabrication boundary validation

17.2 Extension Categories

Permitted extensions:

- Additional required structural fields
- Additional optional traceability signals
- Additional validator types
- Expanded module coverage

Prohibited extensions:

- Governance adequacy scoring
- Risk grading
- Compliance evaluation
- Policy sufficiency inference

The engine must remain structurally bounded.

17.3 Backward Compatibility Model

Let:

R_1 = rule version 1.1
R_2 = rule version 1.2

Compatibility condition:

$$R_2.\text{required} \supseteq R_1.\text{required}$$

Historical freeze packs remain valid under their original rule version.

Re-inspection under a new rule version generates a new freeze pack.

Rule evolution MUST NOT retroactively invalidate historical freezes.

18. Structural Non-Authority Statement

The Rule Engine Formalization:

- Defines structural inspection enforcement only.
- Does not define governance requirements.
- Does not implement standards.
- Does not certify compliance.
- Does not evaluate adequacy.
- Does not assign maturity levels.

Hash stability confirms structural immutability only.

Authority remains external to the rule engine and must be exercised by human governance actors.

Annex A — Terminology Alignment (Informative)

A.1 Purpose

This annex aligns terminology used in the Rule Engine Formalization with established governance, standards, and validation language.

The purpose is interpretive clarity — not normative mapping.

A.2 Core Terms

Rule Engine

A deterministic structural validation and correction system operating under ID_3 inspection posture.

It enforces structural constraints prior to canonical promotion.

It does not evaluate governance adequacy.

Required Structural Constraint

A field or structure that MUST exist and validate successfully for freeze eligibility.

Failure blocks canonical promotion.

Optional Structural Signal

A non-blocking field contributing to diagnostic completeness scoring.

Optional signals improve traceability but do not affect freeze eligibility.

Non-Fabrication Constraint

A structural invariant prohibiting the introduction of semantic content not present in the source artifact.

Violation triggers escalation and posture reversion.

Canonical Artifact

A deterministically ordered, structurally normalized JSON artifact eligible for freeze and integrity anchoring.

Freeze

The explicit stabilization of canonical artifacts under ID_3 posture, followed by integrity anchoring.

Freeze is structural.

It is not evaluative.

Integrity Anchor

A deterministic cryptographic hash (SHA-256) applied to canonical artifacts to ensure immutability and reproducibility.

Completeness Score

A diagnostic metric reflecting optional field presence.

It does not measure governance maturity or adequacy.

A.3 Alignment with ISO/IEC Governance Language

Rule Engine Term	ISO/IEC Analogous Concept	Clarification
Required Constraint	Mandatory control field	Structural only
Optional Signal	Supporting documentation element	Non-evaluative
Validation	Conformance check	Structural conformance only
Freeze	Baseline snapshot	Not approval
Integrity Anchor	Configuration baseline hash	Not certification
Escalation	Nonconformity requiring review	Revert to ID_2 posture

The Rule Engine does not implement ISO controls.
It implements structural discipline only.

Annex B — Validator Definitions (Informative)

B.1 Validator Type Registry

The following validator types are formally defined.

---

presence

Ensures field existence.

```
{
  "validator_type": "presence",
  "field": "session.schema_version",
```

```
  "required": true
}
```

Failure severity: ERROR

---

type

Ensures correct JSON type.

Supported types:

- string
- number
- boolean
- array
- object
- datetime_utc
- enum
- regex_match

Failure severity: ERROR

---

shape

Ensures array/object structural consistency.

Failure severity: ERROR (correctable if metadata-bound)

---

placeholder_check

Detects placeholder patterns.

Failure severity: WARNING (correctable if deterministic source exists)

---

prohibition

Detects disallowed semantic derivation.

Failure severity: ESCALATION_REQUIRED

B.2 datetime_utc Validator Specification

`datetime_utc` MUST:

- Be ISO 8601 format
- Include explicit UTC indicator (`Z`)
- Not derive from filesystem metadata

Example valid:

```
2026-02-10T07:30:00Z
```

Example invalid:

```
2026-02-10 07:30:00
02/10/2026
```

B.3 Deterministic Ordering Rule

Canonical ordering MUST:

- Sort object keys lexicographically
- Maintain stable nested object ordering
- Avoid runtime-generated fields

Deterministic ordering is prerequisite for integrity anchoring.

Annex C — Algorithm References (Informative)

C.1 Hashing Algorithm

ID_3 integrity anchoring uses:

SHA-256 (Secure Hash Algorithm 256-bit)

Properties:

- Collision-resistant

- Deterministic
- Widely standardized
- Suitable for inspection reproducibility

No proprietary algorithm permitted.

C.2 Canonicalization Requirements

Before hashing:

- JSON must be normalized
- Whitespace must be standardized
- Encoding must be UTF-8
- Ordering must be deterministic

Failure to canonicalize invalidates reproducibility guarantees.

Annex D — Worked Example (Baseline vs Variant Enforcement Trace)

D.1 Scenario Overview

Two ID_2 session artifacts are submitted for inspection:

- Baseline session
- Variant session

Both contain:

- session.json
- module_03.json

Rule specification version:

```
rule_id: CSC_SESSION_COMPLETENESS_V1_1
schema_version: rule_v1_1
```

D.2 Initial Verification

Baseline

Detected:

- Placeholder session_id
- Datetime stored as local time
- Optional fields present but empty
- No required-field failures

Severity classification:

- placeholder_check → WARNING
- datetime format → ERROR (correctable)
- optional emptiness → INFO

Freeze eligibility: BLOCKED (pending correction)

Variant

Detected:

- Valid session_id
- Correct UTC timestamps
- All required fields present

Severity classification:

- optional emptiness → WARNING

Freeze eligibility: CONDITIONAL (pending baseline alignment)

D.3 Correction Phase

Baseline corrections applied:

- Deterministic session_id replacement
- Datetime normalization to ISO 8601 UTC
- Array coercion where required

No semantic content altered.

Re-verification:

- All required constraints PASS
- Optional warnings remain diagnostic

D.4 Canonical Promotion

Canonical artifacts generated:

- session_baseline.canonical.json
- session_variant.canonical.json
- module_03_baseline.canonical.json
- module_03_variant.canonical.json

Deterministic ordering applied.

Inspection metadata fields added.

D.5 Freeze and Hash

Freeze declared:

```
inspection_status = frozen_under_ID_3
status = final
frozen_at_utc = 2026-02-26T08:29:29Z
```

Integrity anchor generated:

```
hashes.txt
SHA-256(session_baseline.canonical.json)
SHA-256(session_variant.canonical.json)
```

Example metadata fields shown for illustration only. Actual freeze metadata structure is defined in the ID_3 Operator release.

D.6 Observations

Baseline required structural correction.

Variant satisfied required constraints without modification.

Freeze eligibility was governed strictly by required constraints.

Optional deficiencies did not block freeze.

No semantic inference occurred.

This example demonstrates deterministic structural enforcement under ID_3 posture.
